AWS CodeDeploy

Introduction

AWS CodeDeploy is a **fully managed deployment service** that automates software deployments to various computing services like **Amazon EC2, AWS Fargate, AWS Lambda, and on-premises servers**.

- **What is AWS CodeDeploy?**
- **Why is it useful for DevOps and CI/CD?**
- **How can we configure AWS CodeDeploy for automatic deployments?**

This guide explains **how AWS CodeDeploy simplifies application deployment** and improves efficiency.

Section 1: Learn

What is AWS CodeDeploy?

AWS CodeDeploy **automates application deployments**, reducing downtime and minimizing manual intervention.

Why Use AWS CodeDeploy?

Feature	**Benefit**
Automated Deployments	Reduces manual work and errors
Zero Downtime	Supports rolling updates and blue-green deployments
Scalability	Deploys applications across thousands of instances

Feature	Benefit
Cross-Platform Support	Works with EC2, Lambda, ECS, and on-premises servers
Seamless Integration	Works with AWS CodePipeline and AWS CodeBuild

AWS CodeDeploy Workflow

Stage	Purpose
Step 1: Define Application	Specify application components
Step 2: Configure Deployment	Choose deployment environment (EC2, Lambda, ECS)
Step 3: Monitor Deployment	Track progress via AWS Console or CloudWatch
Step 4: Rollback if Necessary	Automatically rollback on failure

Deployment Strategies in AWS CodeDeploy

Strategy	Description
In-Place Deployment	Updates existing instances without adding new ones
Rolling Deployment	Updates instances in batches to minimize downtime
Blue-Green Deployment	Deploys new code to a separate environment before switching traffic

Strategy	Description
Canary Deployment	Gradually releases new code to a small percentage of users

Interesting Fact

Companies like **Amazon, Airbnb, and Netflix** use AWS CodeDeploy to ensure **reliable and automated deployments with zero downtime**.

Section 2: Practice

1. Setting Up AWS CodeDeploy for EC2 Instances

Step 1: Create an EC2 Instance

1. Open **AWS Management Console** → Navigate to **EC2**.
2. Click **Launch Instance** → Choose an **Amazon Linux 2 AMI**.
3. Configure **Security Group** to allow **HTTP and SSH traffic**.
4. Click **Launch** and connect via SSH.

Step 2: Install CodeDeploy Agent on EC2

Connect to the EC2 instance and run:

```
sudo yum update -y
sudo yum install ruby
sudo yum install wget
cd /home/ec2-user
wget
https://aws-codedeploy-<region>.s3.<region>.amazonaws.com/latest/install
chmod +x ./install
sudo ./install auto
```

```
sudo service codedeploy-agent start
```

Step 3: Create an Application in AWS CodeDeploy

1. Open **AWS CodeDeploy Console** → Click **Create Application**.
2. Enter **Application Name** → Select **Compute Platform: EC2/On-Premises**.
3. Click **Create Deployment Group** → Select **Deployment Type: Rolling or Blue-Green**.
4. Attach an **IAM Role** with AWSCodeDeployRole permissions.

Step 4: Define the appspec.yml File

Create an appspec.yml file in your Git repository:

```
version: 0.0
os: linux
files:
  - source: /
    destination: /var/www/html/
hooks:
  BeforeInstall:
    - location: scripts/before_install.sh
      timeout: 300
  AfterInstall:
    - location: scripts/after_install.sh
      timeout: 300
```

Step 5: Deploy the Application

1. Open **AWS CodeDeploy** → Click **Create Deployment**.
2. Select **Application & Deployment Group**.
3. Choose **GitHub, S3, or CodeCommit** as the source.
4. Click **Deploy Now** → Monitor progress in **AWS CodeDeploy Console**.

Why is this useful?

- Automates **application updates and rollbacks** on EC2.

---

2. Deploying a Serverless Application Using AWS CodeDeploy

AWS CodeDeploy also supports **AWS Lambda deployments**.

Step 1: Configure AWS CodeDeploy for Lambda

1. Open **AWS CodeDeploy** → Click **Create Application**.
2. Choose **Compute Platform: AWS Lambda**.
3. Create a new **Deployment Group** → Select Lambda Function.

Step 2: Modify the appspec.yml for Lambda

```
version: 0.0
Resources:
 - TargetService:
    Type: AWS::Lambda::Function
    Properties:
      Name: MyLambdaFunction
      Alias: live
      CurrentVersion: 1
```

Step 3: Deploy the Lambda Function

1. Upload the Lambda function package to **Amazon S3**.
2. Open **AWS CodeDeploy** → Click **Create Deployment**.
3. Select **Lambda Application** → Choose the Lambda function and S3 location.
4. Click **Deploy Now**.

Why is this useful?

- Enables **zero-downtime deployments for serverless applications**.

Section 3: Know More

Frequently Asked Questions

1. **What is the purpose of AWS CodeDeploy?**
    - It automates **application deployment** to EC2, Lambda, ECS, and on-prem servers.
2. **What is the difference between CodeDeploy and CodePipeline?**
    - **AWS CodeDeploy** focuses on **deployment**, while **AWS CodePipeline** manages the **entire CI/CD workflow**.
3. **What happens if a deployment fails?**
    - AWS CodeDeploy can **automatically rollback** to the previous stable version.
4. **Can AWS CodeDeploy work with GitHub?**
    - Yes! It can fetch deployment files from **GitHub, S3, and AWS CodeCommit**.
5. **How do I monitor AWS CodeDeploy logs?**
    - Logs are available in **AWS CloudWatch** and **CodeDeploy Console**.

Conclusion

AWS CodeDeploy **automates software deployment, ensuring reliability and zero downtime**.

Start by **creating an EC2 instance, setting up the CodeDeploy agent, defining an appspec.yml file, and automating the deployment process**, and soon you'll be **deploying applications seamlessly in AWS Cloud!**